

Smart Transportation Network Planning and Resilience Analysis

Akshaya
Department of Mathematics
Indian Institute of Technology Kanpur

1 Problem Statement

The objective of this project is to design a smart transportation network connecting 500 cities using graph algorithms. Each city is represented as a vertex and each road is represented as a weighted edge. The project aims to reduce construction cost, identify important cities, find alternate routes during disasters, manage traffic congestion, and detect critical parts of the network.

2 Objectives

The main objectives of this project are:

- Construct a connected transportation network with 500 cities.
- Minimize construction cost using a Minimum Spanning Tree.
- Identify strategically important cities based on connectivity.
- Compute alternate routes when cities or roads become unavailable.
- Simulate traffic congestion using dynamic edge weights.
- Detect critical cities and roads that affect network connectivity.

3 Algorithms Used

The following graph algorithms are used in this project.

- **Kruskal's Algorithm** – Used to construct the Minimum Spanning Tree (MST).
- **Disjoint Set Union (DSU)** – Used with Kruskal's algorithm for efficient cycle detection.
- **Dijkstra's Algorithm** – Used to find the shortest path between two cities.
- **Tarjan's Algorithm** – Used to identify articulation points (critical cities) and bridges (critical roads).
- **Degree Centrality** – Used to identify the most connected and strategically important cities.

4 Implementation Details

The project is implemented in C++ using an object-oriented approach. The transportation network is stored using an adjacency list. A random connected graph consisting of 500 cities and 2000 roads is generated, where each road is assigned a random weight representing construction cost or travel distance.

The implementation performs the following five tasks:

1. Minimum Spanning Tree Optimization
2. Strategic City Identification
3. Disaster Recovery Routing
4. Traffic-Aware Smart Routing
5. Critical Infrastructure Analysis

5 Results

The implementation successfully completed all the required tasks.

- Generated a connected weighted graph with 500 cities.
- Reduced the overall infrastructure cost using the Minimum Spanning Tree.
- Ranked cities based on their connectivity.
- Computed alternate shortest paths after removing cities or roads.
- Simulated traffic congestion using dynamic edge weights.
- Identified critical cities and roads using Tarjan's algorithm.

The program prints the output for each task through the console.

6 Complexity Analysis

Algorithm	Time Complexity
Graph Generation	$O(E)$
Kruskal's Algorithm	$O(E \log E)$
Dijkstra's Algorithm	$O(E \log V)$
Degree Centrality	$O(V + E)$
Tarjan's Algorithm	$O(V + E)$

7 Conclusion

This project demonstrates the practical use of graph algorithms in transportation planning. Kruskal's algorithm helps reduce construction cost, Dijkstra's algorithm provides efficient route planning, and Tarjan's algorithm identifies critical infrastructure that should be protected. The project combines these algorithms to build a transportation network that is cost-effective, reliable, and resilient to failures.

8 References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, *Introduction to Algorithms*, MIT Press.
2. Steven S. Skiena, *The Algorithm Design Manual*, Springer.
3. Lecture Notes, Graph Theory Course, IIT Kanpur.

A Complete Source Code

Listing 1: C++ Implementation of Smart Transportation Network Planning and Resilience Analysis

```
1 #include <iostream>
2 #include <vector>
3 #include <queue>
4 #include <set>
5 #include <map>
6 #include <random>
7 #include <algorithm>
8 #include <iomanip>
9 #include <fstream>
10 #include <limits>
11
12 class SmartCityNetwork {
13 private:
14     struct DSU {
15         std::vector<int> parent, rank;
16         DSU(int n) {
17             parent.resize(n + 1);
18             rank.assign(n + 1, 0);
19             for (int i = 0; i <= n; i++) parent[i] = i;
20         }
21         int find(int i) {
22             if (parent[i] == i) return i;
23             return parent[i] = find(parent[i]);
24         }
25         void unite(int i, int j) {
26             int root_i = find(i);
27             int root_j = find(j);
28             if (root_i != root_j) {
29                 if (rank[root_i] < rank[root_j]) parent[root_i] =
                    root_j;
30                 else if (rank[root_i] > rank[root_j]) parent[
                    root_j] = root_i;
31                 else {
32                     parent[root_j] = root_i;
33                     rank[root_i]++;
34                 }
35             }
36         }
37     };
38 }
```

```

36     }
37 };
38
39 struct Edge {
40     int u, v;
41     double w;
42     bool operator<(const Edge& other) const {
43         return w < other.w;
44     }
45 };
46
47 int V;
48 std::vector<std::vector<std::pair<int, double>>> adj;
49 std::vector<Edge> edges;
50 std::mt19937 rng;
51 const double INF = std::numeric_limits<double>::infinity();
52
53 void addEdge(int u, int v, double w) {
54     adj[u].push_back({v, w});
55     adj[v].push_back({u, w});
56     edges.push_back({u, v, w});
57 }
58
59 void runDijkstra(int src, int dest, int skipNode = -1, std::
    pair<int, int> skipEdge = {-1, -1}, std::map<std::pair<int
    ,int>, double>* trafficMap = nullptr) {
60     std::vector<double> dist(V, INF);
61     std::vector<int> parent(V, -1);
62     std::priority_queue<std::pair<double, int>, std::vector<
        std::pair<double, int>>, std::greater<std::pair<double
        , int>>> pq;
63
64     dist[src] = 0;
65     pq.push({0, src});
66
67     while (!pq.empty()) {
68         double d = pq.top().first;
69         int u = pq.top().second;
70         pq.pop();
71
72         if (u == dest) break;
73         if (d > dist[u]) continue;
74
75         for (const auto& edge : adj[u]) {
76             int v = edge.first;
77             double w = edge.second;
78
79             if (v == skipNode) continue;
80             if ((u == skipEdge.first && v == skipEdge.second)
                || (u == skipEdge.second && v == skipEdge.
                    first)) continue;

```

```

81
82         if (trafficMap != nullptr) {
83             std::pair<int, int> key = {std::min(u, v),
84                                     std::max(u, v)};
85             if (trafficMap->count(key)) {
86                 w *= (*trafficMap)[key];
87             }
88         }
89
90         if (dist[u] + w < dist[v]) {
91             dist[v] = dist[u] + w;
92             parent[v] = u;
93             pq.push({dist[v], v});
94         }
95     }
96
97     if (dist[dest] == INF) {
98         std::cout << "> No valid route available after
99         disaster.\n";
100         return;
101     }
102
103     std::cout << "> Distance: " << std::fixed << std::
104         setprecision(2) << dist[dest] << " km | Path: ";
105     std::vector<int> path;
106     for (int curr = dest; curr != -1; curr = parent[curr])
107         path.push_back(curr);
108     std::reverse(path.begin(), path.end());
109     for (int node : path) std::cout << node << " ";
110     std::cout << "\n";
111 }
112
113 void dfsTarjan(int u, int p, std::vector<int>& tin, std::
114     vector<int>& low, int& timer,
115     int& maxCompsCity, int& criticalCity, std::
116     vector<std::pair<int, int>>& bridges) {
117
118     tin[u] = low[u] = timer++;
119     int children = 0;
120     int splits = (p != -1) ? 1 : 0;
121
122     for (const auto& edge : adj[u]) {
123         int v = edge.first;
124         if (v == p) continue;
125
126         if (tin[v] != -1) {
127             low[u] = std::min(low[u], tin[v]);
128         } else {
129             children++;

```

```

125         dfsTarjan(v, u, tin, low, timer, maxCompsCity,
126                 criticalCity, bridges);
127         low[u] = std::min(low[u], low[v]);
128
129         if (low[v] >= tin[u] && p != -1) {
130             splits++;
131         }
132
133         if (low[v] > tin[u]) {
134             bridges.push_back({std::min(u, v), std::max(u,
135                 v)});
136         }
137     }
138
139     if (p == -1) {
140         splits = children;
141     }
142
143     if (splits > maxCompsCity) {
144         maxCompsCity = splits;
145         criticalCity = u;
146     }
147 }
148 public:
149     SmartCityNetwork(int vertices) : V(vertices), adj(vertices),
150         rng(42) {}
151
152     void generateGraph(int numEdges) {
153         std::uniform_real_distribution<double> weightDist(10.0,
154             150.0);
155         std::set<std::pair<int, int>> used;
156
157         for (int i = 1; i < V; i++) {
158             std::uniform_int_distribution<int> nodeDist(0, i - 1)
159                 ;
160             int u = i;
161             int v = nodeDist(rng);
162             double w = weightDist(rng);
163
164             used.insert({std::min(u, v), std::max(u, v)});
165             addEdge(u, v, w);
166         }
167
168         std::uniform_int_distribution<int> allNodes(0, V - 1);
169         int added = V - 1;
170
171         while (added < numEdges) {
172             int u = allNodes(rng);
173             int v = allNodes(rng);

```

```

171         if (u != v) {
172             std::pair<int, int> edgeKey = {std::min(u, v),
173             std::max(u, v)};
174             if (used.find(edgeKey) == used.end()) {
175                 double w = weightDist(rng);
176                 used.insert(edgeKey);
177                 addEdge(u, v, w);
178                 added++;
179             }
180         }
181     }
182 }
183
184 void task1_MST() {
185     std::cout << "\n--- TASK 1: MST OPTIMIZATION ---\n";
186     double totalCost = 0;
187     for (const auto& e : edges) totalCost += e.w;
188
189     std::vector<Edge> tempEdges = edges;
190     std::sort(tempEdges.begin(), tempEdges.end());
191
192     DSU dsu(V);
193     double mstCost = 0;
194     std::vector<std::pair<int, int>> mstRoads;
195
196     for (const auto& e : tempEdges) {
197         if (dsu.find(e.u) != dsu.find(e.v)) {
198             dsu.unite(e.u, e.v);
199             mstCost += e.w;
200             mstRoads.push_back({e.u, e.v});
201         }
202     }
203
204     double saved = ((totalCost - mstCost) / totalCost) *
205     100.0;
206     std::cout << std::fixed << std::setprecision(2);
207     std::cout << "> Original Cost = " << totalCost << "\n";
208     std::cout << "> MST Cost = " << mstCost << "\n";
209     std::cout << "> Cost Saved = " << saved << "%\n";
210
211     std::cout << "> Selected Roads:\n";
212     // Printing only the first 10 to avoid console spam,
213     // similar to real-world log truncations
214     for (size_t i = 0; i < std::min((size_t)10, mstRoads.size
215     ()); i++) {
216         std::cout << "(" << mstRoads[i].first << "," <<
217         mstRoads[i].second << ")\n";
218     }
219     std::cout << "[All " << mstRoads.size() << " structural
220     spans successfully written to path buffer]\n";

```

```

216         std::ofstream fout("mst_edges.txt");
217         if (fout.is_open()) {
218             for (const auto& r : mstRoads) {
219                 fout << r.first << " " << r.second << "\n";
220             }
221             fout.close();
222         }
223     }
224 }
225
226 void task2_StrategicCities() {
227     std::cout << "\n--- TASK 2: STRATEGIC CITY IDENTIFICATION\n";
228     std::vector<std::pair<int, int>> deg;
229     for (int i = 0; i < V; i++) {
230         deg.push_back({adj[i].size(), i});
231     }
232
233     std::sort(deg.rbegin(), deg.rend());
234
235     for (int i = 0; i < 10 && i < V; i++) {
236         std::cout << "> Rank " << i + 1 << " City " << deg[i]
237             .second << " | Degree " << deg[i].first << "\n";
238     }
239 }
240
241 void task3_Disaster(int src, int dest, int deadCity, int
242     deadRoadU, int deadRoadV) {
243     std::cout << "\n--- TASK 3: DISASTER RECOVERY ROUTING\n";
244     std::cout << "Simulating destruction of City " <<
245         deadCity << ":\n";
246     runDijkstra(src, dest, deadCity);
247
248     std::cout << "Simulating destruction of Road (" <<
249         deadRoadU << ", " << deadRoadV << "):\n";
250     runDijkstra(src, dest, -1, {deadRoadU, deadRoadV});
251 }
252
253 void task4_Traffic(int src, int dest) {
254     std::cout << "\n--- TASK 4: TRAFFIC-AWARE SMART ROUTING\n";
255
256     std::map<std::pair<int,int>, double> trafficWeights;
257     std::uniform_real_distribution<double> mult(1.0, 3.0);
258
259     for (const auto& e : edges) {
260         trafficWeights[{std::min(e.u, e.v), std::max(e.u, e.v)
261             }]} = mult(rng);
262     }
263 }

```



```

259     std::cout << "Normal Route:\n";
260     runDijkstra(src, dest);
261
262     std::cout << "Traffic Route (Static Congestion Applied):\n";
263     runDijkstra(src, dest, -1, {-1, -1}, &trafficWeights);
264 }
265
266 void task5_Critical() {
267     std::cout << "\n--- TASK 5: CRITICAL INFRASTRUCTURE
268     ANALYSIS ---\n";
269
270     std::vector<int> tin(V, -1), low(V, -1);
271     int timer = 0;
272     int maxCompsCity = 1;
273     int criticalCity = -1;
274
275     std::vector<std::pair<int, int>> bridges;
276     dfsTarjan(0, -1, tin, low, timer, maxCompsCity,
277         criticalCity, bridges);
278
279     if (maxCompsCity > 1) {
280         std::cout << "> Most Critical City:\nCity " <<
281             criticalCity << "\n";
282         std::cout << "> Disconnected Components Created:\n"
283             << maxCompsCity << "\n";
284     } else {
285         std::cout << "> Most Critical City: None (Network is
286             strongly connected)\n";
287     }
288
289     if (!bridges.empty()) {
290         std::cout << "> Most Critical Road:\n(" << bridges
291             [0].first << "," << bridges[0].second << ")\n";
292     } else {
293         std::cout << "> Most Critical Road: None (Graph is 2-
294             edge-connected)\n";
295     }
296 }
297 };
298
299 int main() {
300     std::ios_base::sync_with_stdio(false);
301     std::cin.tie(NULL);
302
303     SmartCityNetwork network(500);
304     network.generateGraph(2000);
305
306     network.task1_MST();
307     network.task2_StrategicCities();
308     network.task3_Disaster(12, 203, 91, 12, 55);

```

```
302     network.task4_Traffic(10, 250);
303     network.task5_Critical();
304
305     return 0;
306 }
```